Python Day 3

클래스 · 파일 · 시스템 연동

HYUNDAI AI Insight Campus · 온디바이스 AI · 프로그래밍 언어(Python) 3/4

2026. 9. 4 (금) 09:00–18:00 · 8교시 · Google Colab

클래스 | 상속 | 파일 입출력 | JSON·CSV | 시스템 연동 | SensorLogger

강사: 이몽룡 · 길동

오늘의 학습 목표

수업이 끝나면 여러분은 다음을 할 수 있습니다

1

클래스 기초

dict 를 졸업하고 데이터+동작을 클래스로 묶는다 (self, __init__)

2

클래스 심화

__repr__·클래스 변수·상속과 super() 를 쓴다

3

파일 입출력

with open 으로 로그를 읽고 쓰며 w/a/r 을 구분한다

4

JSON·CSV

데이터를 저장·복원해 세션이 끝나도 남는 프로그램을 만든다

5

시스템 연동

pathlib·subprocess 를 쓰고 시리얼 수집 루프의 구조를 읽는다

6

종합

SensorLogger — 수집→저장→복원→리포트를 완성한다

시간표

8교시 × 50분 · 점심 12:00–13:00 · 모든 실습은 Google Colab

교시	시간	주제	형태	애니메이션
1	09:00–09:50	클래스 기초 — dict 를 졸업하기	강의 + 실습	D3-1 클래스·인스턴스 모델
2	10:00–10:50	클래스 심화 — repr·클래스 변수·상속	강의 + 실습	D3-1 ⑤⑥, D3-2 상속·메서드 탐색
3	11:00–11:50	파일 입출력	강의 + 실습	D3-3 with open 파일 흐름
4	13:00–13:50	JSON 과 CSV	강의 + 실습	D3-4 JSON 직렬화 왕복
5	14:00–14:50	시스템 연동 — os·subprocess·시리얼	강의 + 실습	D3-5 시리얼 스트림, 04 파이프라인
6	15:00–15:50	종합 설계 — SensorLogger	강의 + 실습	D3-1, D3-4, 03 (메서드 호출 스택)
7	16:00–16:50	종합 실습 — 저장되는 센서 로거	실습	D3-4, D2-5 트리 뷰어
8	17:00–17:50	정리·퀴즈·제출	평가	해설용 전체

오늘 사용하는 학습 도구 3종

예측 → 애니메이션에서 확인 → Colab에서 코드로 검증 → 자동 채점 — Day 1·2와 동일

애니메이션 9개

…/anim/index.html

- 신규 5개(day3) + 재사용 4개(day1·day2)
- 프리셋 버튼으로 시나리오 재생
- D3-1 의 self 화살표, D3-3 의 'w 가 지우는 순간'
- 학생도 같은 주소로 접속 — 실습 중 직접 조작

Colab 워크시트

Day3_worksheet.ipynb

- 설정 셀에 Day 2 파서·데이터 내장
- 교시별 따라하기 → TODO → ✅ 자동 채점 → 🔍 점검
- 과제 A 는 저장→복원→이어쓰기 시나리오를 채점
- 제출: 노트북 + siteA.json

개념 워크시트

…/anim/worksheet.html

- 교시별 10문항 × 7 = 70문항 (Day 3 전용)
- 채점 후 정오·해설·확인할 애니메이션 표시
- 제출 → 오답 정리 대화상자 → 캡처로 노션에
- 결과 코드를 Colab 제출 요약 셀에 입력

예측 → 애니메이션 확인 → 코드로 검증 → 자동 채점

애니메이션 — 신규 5개 + 재사용 4개

다크 테마 공통 디자인 · 프리셋 버튼 → 캔버스 → 상태 패널 → 설명

D3-1 클래스·인스턴스 메모리 모델 — 1·2·6교시
설계도 vs 인스턴스 · self 화살표 · 조회 경로 · dict 비교

D3-2 상속과 메서드 탐색 — 2교시
자식→부모 탐색 · 오버라이드 · super() · super 누락 사고

D3-3 with open 파일 흐름 — 3·6교시
프로그램→버퍼→디스크 · 'w'가 지우는 순간 · 자동 close

D3-4 JSON 직렬화 왕복 — 4·7교시
dict↔문자열↔파일 · 타입 복원 · 인스턴스 ✕ → to_dict

D3-5 시리얼 스트림 시뮬레이터 — 5교시
readline 절단 · 손상·조각·빈 줄 · 파서가 붙는 구조

03 제어 흐름 추적기 (day1) — 6교시
[Day 2] 함수 호출과 스택 — 메서드 호출도 같은 프레임

04 문자열 파싱 파이프라인 (day1) — 5교시
스트림에서 잘라낸 한 줄이 들어가는 곳

D2-5 중첩 자료구조 트리 뷰어 (day2) — 7교시
저장할 JSON 구조 설계 확인

D2-6 자료구조 결정 트리 (day2) — 1교시
도입 — 'dict 로 부족해지는 순간' 토론

제목 클릭 시 열림 · Day 3 목차: https://devops0906.github.io/AI_CAMPUS/python-day3/anim/index.html

워크시트 사용법

Day 1·2와 동일 — Colab 노트북 하나로 실습·채점·제출

Colab 워크시트 — 교시당 한 섹션

1	⚙️ 설정	이름 입력 후 가장 먼저 실행 (parse_line·데이터 제공)
2	따라하기	실행만 하고 출력을 읽는다
3	TODO 1~4	raise NotImplementedError 줄을 지우고 작성
4	✅ 자동 채점	몇 번이든 재실행 — PASS n/4 를 남긴다
5	🔍 자기 점검	드롭다운으로 예측 → 애니메이션에서 확인
6	막힌 점 한 줄	텍스트 셀에 기록

개념 워크시트 (HTML · Day 3)

- 교시별 10문항 · 채점하기 전까지 답 수정 가능
- 채점 후 정오 + 해설 + 확인할 애니메이션 프리셋
- 제출 → 오답 정리 대화상자 → 캡처로 노션에 붙여넣기
- 결과 코드 7개를 Colab '제출 요약' 셀에 입력

https://devops0906.github.io/AI_CAMPUS/python-day3/anim/worksheet.html

제출

```
이름: 홍길동
1교시      자동 채점 4/4
...
7교시-B    자동 채점 4/4
합계 32/32
siteA.json: 있음 ✅
```

온라인 자료 링크

브라우저에서 바로 열립니다 — 설치·다운로드 없음

기본 주소 (Day 3)

https://devops0906.github.io/AI_CAMPUS/python-day3/anim/

index.html	Day 3 애니메이션 목차 (신규 5 + 재사용 4 링크)
worksheet.html	Day 3 개념 워크시트 (70문항 · 제출 · 캡처)

신규 애니메이션 (day3 폴더)

1·2교시 D3-1_클래스_인스턴스_모델.html

2교시 D3-2_상속_메서드탐색.html

3교시 D3-3_파일_입출력_흐름.html

4교시 D3-4_JSON_직렬화_왕복.html

5교시 D3-5_시리얼_스트림.html

재사용: 03_추적기 · 04_파이프라인 (day1) · D2-5_트리_뷰어 · D2-6_결정_트리 (day2)

1 클래스 기초 — dict 를 졸업하기

09:00–09:50

학습 목표

- dict 레코드의 한계(키 오타, 동작 분리)를 체감한다
- class · __init__ · 인스턴스 속성 · 메서드 정의
- self = "지금 이 인스턴스" 를 설명한다
- 인스턴스 속성은 서로 독립임을 안다

🎬 **D3-1 클래스·인스턴스 모델**

- ① 설계도만 — 데이터 0개 → ② 인스턴스 생성
- ③ 독립 → ④ self 화살표가 인스턴스마다 이동
- ⑦ dict 오타 vs 클래스 오타 비교
- 도입 토론: D2-6 결정 트리 — dict 로 부족한 순간

핵심 개념

첫 클래스

```
class SensorReading:
    def __init__(self, node_id, t, h, d):
        self.node_id = node_id; self.t = t ...
    def is_alert(self):
        return self.t > 35
```

self 정체

```
r1.is_alert()  ==  SensorReading.is_alert(r1)
호출한 인스턴스가 self 로 자동 전달
```

오타의 운명

```
d["Temp"] → KeyError (실행해야 터짐, 힌트 없음)
r1.tmp → AttributeError (즉시·명확)
```

독립

```
r2.t = 37.5  # r1 은 영향 없음 — 속성 상자는 각자
```

1 클래스 기초 — 실습

Colab 워크시트 TODO

1 TODO 1-1 SensorReading
__init__ 으로 속성 4개 저장

2 TODO 1-2 is_alert
self.t > 35 (경계 35.0 은 False)

3 TODO 1-3 level
Day 1 판정 규칙을 메서드로 이식

4 TODO 1-4 self 정체 (심화)
same = (r1.is_alert() == SensorReading.is_alert(r1))

✅ **자동 채점** 속성 4개 · is_alert 경계 · level 5단계 · same (4항목)

🔍 자기 점검 (예측 → 애니메이션 → 코드)

- self 는? → 지금 메서드를 호출한 인스턴스
- r1.tmp 오타 → AttributeError

체크포인트 · 강사 확인

- "클래스는 설계도, 인스턴스는 데이터"를 자기 말로 하는가
- __init__ 안에서 self. 없이 대입한 학생(지역 변수) 교정
- 메서드에 self 를 빼먹은 학생 — TypeError 메시지 읽기

2 클래스 심화 — repr·클래스 변수·상속

10:00–10:50

학습 목표

- __repr__ 로 읽기 좋은 객체를 만든다
- 클래스 변수(공유) vs 인스턴스 변수(각자), 조회 순서
- 상속·오버라이드·super() 를 쓴다
- 가변 클래스 변수 함정을 안다

🎬 **D3-1 ⑤⑥ · D3-2 상속**

- D3-1 ⑤ 조회 경로(점선) → ⑥ 클래스 변수 전파
- D3-2 ② 자식에 없으면 부모로 → ③ 오버라이드는 자식에서 멈춤
- ④ super() 가 부모 __init__ 을 먼저 실행
- ⑤ super 누락 사고 — AttributeError 재현

핵심 개념

__repr__

```
def __repr__(self):
    return f"Reading({self.node_id}, T={self.t})"
print(r)  # 디버깅의 절반은 좋은 repr
```

클래스 변수

```
ALERT_T = 35    # 설계도에 한 부
조회: 인스턴스 → 클래스 순서
SensorReading.ALERT_T = 30  # 전체 전파
```

상속

```
class CalibratedReading(SensorReading):
    def __init__(self, node_id, t, offset):
        super().__init__(node_id, t)  # 부모 먼저
        self.offset = offset
```

함정

자식 `__init__` 에서 `super()` 누락 → `t` 가 없다 `(AttributeError)`

2 클래스 심화 — 실습

Colab 워크시트 TODO

1. **TODO 2-1 __repr__**
   Reading(ESP32-01, T=25.3) 정확 형식

2. **TODO 2-2 전파 + 복구**
   ALERT_T=30 → 판정 확인 → 반드시 35 복구

3. **TODO 2-3 CalibratedReading**
   super().__init__ + is_alert 오버라이드

4. **TODO 2-4 가변 클래스 변수 (심화)**
   Tally.log 공유를 실험으로 재현

✅ **자동 채점** repr 형식 · 전파+복구 · 상속 3검사 · shared [1,2] (4항목)

🔍 자기 점검 (예측 → 애니메이션 → 코드)

- super 누락 → 부모 속성이 없어 AttributeError

체크포인트 · 강사 확인

- 조회 순서(인스턴스→클래스)를 말로 설명하는가
- r1.ALERT_T = 30 대입(인스턴스 변수 생성)과 클래스 대입을 구분하는가
- "상속은 2단까지만" 실무 권고 전달

3 파일 입출력

11:00–11:50

학습 목표

- with open(path, mode, encoding) 패턴을 기본으로 쓴다
- w(지움) / a(이어쓰기) / r(없으면 예외) 구분
- 한 줄씩 읽기 + strip 습관
- close 와 버퍼 — with 의 이유

🎬 **D3-3 with open 파일 흐름**

- ① 'w' 가 지우는 순간 — write 전에 이미 빈 파일
- ② 'a' 이어쓰기 → ③ 한 줄씩 읽기
- ④ close 없이 종료 — 버퍼 유실
- ⑤ 없는 파일 — 6교시 load 방어의 원형

핵심 개념

쓰기 3박자

```
with open(p, "w", encoding="utf-8") as f:
    f.write("T=25.3,H=60\n")   # 개행은 직접
# with 블록 끝 → 자동 close → 버퍼→디스크
```

모드

```
"w"  여는 순간 기존 내용 삭제
"a"  뒤에 이어쓰기 (로그 표준)
"r"  기본값, 없으면 FileNotFoundError
```

읽기

```
with open(p, encoding="utf-8") as f:
    for line in f:          # 메모리 효율
        print(line.strip())  # \n 제거
```

방어

```
try: open(...) except FileNotFoundError: lines = []
```

3 파일 입출력 — 실습

Colab 워크시트 TODO

1 TODO 3-1 로그 저장
LOG_TEXT → day1_log.txt (10줄 검증)

2 TODO 3-2 append_log
"a" 모드 + line + "₩n"

3 TODO 3-3 파일에서 파싱
for line in f + parse_line → good 8 / bad 2

4 TODO 3-4 safe_write (심화)
이미 있으면 False — 덮어쓰기 방지

✅ **자동 채점** 10줄 · append 2회 · 8/2 · safe_write (4항목)

🔍 자기 점검 (예측 → 애니메이션 → 코드)

- open(p, "w") → 여는 순간 지운다

체크포인트 · 강사 확인

- "w 로 소중한 로그를 연" 사고 시나리오를 말하는가
- f.write 에 개행을 빼먹은 학생 — 한 줄로 붙는 출력 관찰
- encoding="utf-8" 을 습관으로 붙이는가

4 JSON 과 CSV

13:00–13:50

학습 목표

- dump/load(파일) vs dumps/loads(문자열) 구분
- 복원하면 타입이 돌아온다 — split 파싱과의 차이
- indent · ensure_ascii=False 저장 습관
- 인스턴스는 저장 불가 → to_dict 패턴 예고

🎬 D3-4 JSON 직렬화 왕복

- ① dumps → ② dump(지름길) → ③ load 타입 복원
- ④ indent · ensure_ascii — 한글 "정문"
- ⑤ 인스턴스 TypeError → to_dict 패턴
- ⑥ loads vs load 2×2 표

핵심 개념

2×2 표

```
        문자열    파일
저장    dumps     dump
복원    loads     load     # s = string
```

파일 왕복

```
json.dump(stats, f, indent=2, ensure_ascii=False)
restored = json.load(f)
restored == stats   # True, float 도 그대로
```

⚠ 인스턴스

```
json.dumps(r)  → TypeError
JSON 이 아는 것: dict/list/str/숫자/bool/None
해법: r.to_dict() — 6교시에서 완성
```

CSV

```
사람·엑셀용 표는 CSV, 프로그램 왕복은 JSON
```

4 JSON 과 CSV — 실습

Colab 워크시트 TODO

1 TODO 4-1 왕복
DAY2_READINGS → readings.json → restored == 비교

2 TODO 4-2 복원본 계산
전체 T 평균 30.47 — float 이라 바로 계산

3 TODO 4-3 CSV
헤더 id,T,H,D + 6행 (newline="")

4 TODO 4-4 ensure_ascii (심화)
기본 vs False 두 문자열 비교

✅ **자동 채점** 왕복+indent · 30.47 · 헤더+7행 · 이스케이프 (4항목)

🔍 자기 점검 (예측 → 애니메이션 → 코드)

- dump vs dumps → 파일 / 문자열

체크포인트 · 강사 확인

- "복원하면 타입이 돌아온다"를 어제 split 과 대비해 말하는가
- loads 에 파일을 넣은 학생 — 인자를 보면 안다
- 저장 옵션 2개(indent, ensure_ascii)를 습관화

5 시스템 연동 — os·subprocess·시리얼

14:00–14:50

학습 목표

- makedirs(exist_ok) · pathlib / 연산자
- subprocess.run — !명령 을 코드 안에서
- 시리얼 수집 루프의 구조를 읽는다 (실행은 장비 주간)
- 빈 줄·손상·조각 처리와 안전 종료

🎬 D3-5 시리얼 스트림 · 04

- ① 정상 → ② 손상 혼입 → ③ 경계 어긋난 조각
- readline 은 ₩n 에서 자를 뿐 — 검증 필수
- ④ timeout 빈 줄 → ⑤ 전체 리허설(무중단)
- 04: 잘라낸 한 줄이 들어가는 곳 (어제의 파서)

핵심 개념

경로

```
os.makedirs("data", exist_ok=True)
p = Path("data") / "log.txt"   # 덧셈 금지
```

subprocess

```
r = subprocess.run([sys.executable, "-c", ...],
        capture_output=True, text=True)
r.stdout   # Jetson 시스템 정보도 이 형태
```

수집 루프

```
while True:
    line = ser.readline().decode().strip()
    if not line: continue       # timeout
    try: t,h,d = parse_line(line)
    except (FrameError, ValueError): bad+=1; continue
```

안전 종료

```
except KeyboardInterrupt: break  # Ctrl-C
```

5 시스템 연동 — 실습

Colab 워크시트 TODO

1. **TODO 5-1 data 폴더**
   makedirs + 저장 + listdir
2. **TODO 5-2 subprocess**
   -c "print(6*7)" → out "42"
3. **TODO 5-3 리플레이**
   STREAM 집계 → 3 · 2 · 1
4. **TODO 5-4 안전 종료 (심화)**
   KeyboardInterrupt → processed 3

✅ **자동 채점** 폴더+파일 · 42 · 3/2/1 · processed (4항목)

🔍 자기 점검 (예측 → 애니메이션 → 코드)

- readline 절단 기준 → 개행 ₩n

체크포인트 · 강사 확인

- "open 을 Serial 로 바꾸면 그대로 장비 코드"라는 연결을 말하는가
- 경로를 문자열 덧셈으로 만든 학생 → pathlib 로
- 빈 줄을 except 로 처리한 학생 — 오류가 아님을 구분

6 종합 설계 — SensorLogger

15:00–15:50

학습 목표

- 요구사항 → 메서드 목록 도출 (라이브 설계)
- to_dict / from_dict(classmethod) 왕복 패턴
- load 의 예외 배치 — 대처할 수 있는 곳에서
- 메서드 호출도 같은 스택 프레임 (self 가 인자)

🎬 **D3-1 · D3-4 · 03**

- D3-4 ⑤ 인스턴스 → dict → 파일 왕복 완성형
- D3-1 ① — from_dict 의 cls = "설계도로 찍는다"
- 03 [Day 2] 함수 호출 — logger.add 프레임에 self
- 라이브 코딩: 메서드 하나 완성할 때마다 즉시 실행

핵심 개념

설계

```
"모으고, 남기고, 불러와 이어서 모은다"
→ add / save / load / stats_by_node / report
```

왕복

```
def to_dict(self): return {"id": ..., "T": ...}
@classmethod
def from_dict(cls, d): return cls(d["id"], ...)
# 복원 시점엔 인스턴스가 없다 → cls 로 찍는다
```

load 예외

```
try: ... json.load(f) ...
except FileNotFoundError:
    self.readings = []    # 첫 실행은 정상 시나리오
```

영속성

```
save → 새 로거 load → add — 세션을 넘어 이어진다
```

6 SensorLogger — 실습

Colab 워크시트 TODO

1 TODO 6-1 to_dict / from_dict
키 id/T/H/D · classmethod 여부까지 채점

2 TODO 6-2 add / save
to_dict 목록을 dump — 파일 구조 검증

3 TODO 6-3 load 시나리오
없으면 빈 리스트 · save 2 → load → add 1 → 3

4 TODO 6-4 stats_by_node (심화)
Day 2 함수의 메서드화 — self.readings

✅ **자동 채점** 왕복+classmethod · 파일 구조 · 시나리오 · stats (4항목)

🔍 자기 점검 (예측 → 애니메이션 → 코드)

- from_dict 가 classmethod 인 이유 → 인스턴스가 없어 cls 로

체크포인트 · 강사 확인

- from_dict 안에서 self 를 쓰는 학생 → cls 교정 (D3-1 ①)
- FileNotFoundError 와 JSONDecodeError 를 구분해 말하는가
- save 를 add 마다 부르는 설계 → "언제 저장?" 토론

7 종합 실습 — 저장되는 센서 로거

16:00–16:50 · 2인 1조 허용, 각자 제출

과제 A · SensorLogger 완성 (필수, 35분)

- SensorReading: 속성4 + __repr__ + is_alert + to_dict + from_dict
- SensorLogger: add · save · load · stats_by_node · report
- 시나리오: siteA.json — 6개 add → save → 새 로거 load → 2개 추가 → 총 8개
- report: f-string 표 + ALERT 노드 ⚠, 컴프리헨션 1회 이상

```
node       avg_T   max_T  alerts
ESP32-01    31.6    37.2       2 ⚠
ESP32-02    33.4    36.9       1 ⚠
ESP32-03    27.5    33.4       0
```

✅ 4항목 — 8개 시나리오 · 파일 구조 · stats 정합 · ⚠ 표시

과제 B · 내보내기와 복구 (선택, 15분)

- export_csv — 헤더 node,T,H,D,alert + 8행
- load_robust: path 시도 → JSONDecodeError 면 .bak → 둘 다 실패 시 []
- broken.json / hopeless.json 준비 셀 제공
- err_counts = Counter() 로 예외 이름별 집계

```
1 [] {'JSONDecodeError': 2, 'FileNotFoundError': 1}
```

막히면: 저장 구조 → D2-5 트리 뷰어 · 왕복 → D3-4 ⑤

빨리 끝낸 조: __len__ 구현 · siteA.json 을 Drive 로 옮겨 진짜 영구 저장 확인

8 정리 · 퀴즈 · 제출

17:00–17:50

15분 — **과제 A 모범 답안 리뷰 (🔒 7-A 셀)**
classmethod·예외 배치 중심 — D3-1 ①, D3-4 ⑤ 로 해설

20분 — **Day 3 퀴즈 (Google Forms, 10문항)**
self → D3-1 · w 모드 → D3-3 · dump/dumps → D3-4 로 해설

5분 — **핵심 정리 · Day 4 예고**
로거에 스트림과 리포트를 붙여 프로젝트 완성

10분 — **제출**
Day3_이름.ipynb + siteA.json + 개념 워크시트 캡처

평가 기준 (Day 3 몫)

퀴즈 문항당 1점	10점
Colab TODO 자동 채점 1~6교시 24 + 과제 A 4 + 과제 B 4	32점
개념 워크시트 오답 정리 캡처(노션) 또는 결과 코드	7×10
노트북 + siteA.json 미제출 시 실습 0점	필수

제출 폴더: 공유 Drive › 온디바이스AI_Python › Day3 › 제출
파일명: Day3_이름.ipynb + siteA.json
개념 워크시트:
…/anim/worksheet.html

오늘의 핵심 정리

각 항목 옆 번호는 다시 볼 애니메이션

클래스	설계도와 데이터 — self 는 "지금 이 인스턴스", 오타는 즉시 AttributeError	D3-1
상속	조회는 인스턴스→클래스, 탐색은 자식→부모 · 자식 __init__ 첫 줄은 super()	D3-2
파일	"w" 는 지우고 "a" 는 잇는다 — with 가 close 를 보장	D3-3
JSON	dump/load 파일 · dumps/loads 문자열 — 복원하면 타입이 돌아온다	D3-4
왕복	인스턴스는 to_dict 로 저장, from_dict(classmethod) 로 복원	D3-4
스트림	readline 은 ₩n 절단뿐 — 검증·건너뛰기·안전 종료가 견고한 수집기	D3-5

Day 4 예고

마무리 프로젝트 — 9/7 (월)

- 오늘의 SensorLogger 에 실시간 스트림 리플레이를 붙입니다
- 통계 리포트 + 간단 시각화(matplotlib)
- 코드 패키징(sensors.py + logger.py) 과 조별 발표
- Jetson·라즈베리파이 실습 주간으로 가는 다리 — open() 을 Serial() 로

오늘 과제 (선택)

- 과제 B 미완이면 완성해 오기 (내일 출발 코드)
- siteA.json 을 Drive 로 옮겨 영구 저장 확인
- D3-4 ⑥ 표를 보지 않고 그려 보기

수고하셨습니다.